

Coding Agents

Intelligent Multi-Agent System for
Information Retrieval and Analysis

Project Report

Prepared by

Mohmad Al-Hmoud
Shreef Tamer

Project Type

AI / NLP / Multi-Agent System

September 3, 2026

Abstract

This project presents a multi-agent system designed to improve the process of retrieving, analyzing, and presenting information from a knowledge base. The system is based on three specialized agents: the Retriever Agent, the Analyst Agent, and the Answer Agent.

Each agent has a clearly defined responsibility. The Retriever Agent is responsible for finding relevant evidence from a vector database using different retrieval strategies. The Analyst Agent processes the retrieved evidence, performs calculations and data analysis, extracts information from tables, compares documents, and requests additional evidence when necessary. Finally, the Answer Agent transforms the verified evidence and analysis into a clear final response with accurate source citations.

The project was developed collaboratively by Mohmad Al-Hmoud and Shreef Tamer. The first phase focused on implementing the Retriever and Analyst agents, while the second phase focused on the Answer Agent and the final system features, including deployment, voice input, image processing, report generation, and monitoring.

Contents

1	Introduction	4
2	Problem Statement	4
3	Project Objectives	5
4	System Architecture	5
5	Retriever Agent	6
5.1	Query Rewriter	6
5.2	Semantic Search	7
5.3	Keyword Search	7
5.4	Metadata Filter	7
5.5	Reranker	8
5.6	Context Selector	8
6	Analyst Agent	8
6.1	Calculator	8
6.2	Table Extractor	8
6.3	Document Comparison	9
6.4	Data Analysis	9
6.5	Feedback Loop	9
7	Answer Agent	10
8	Final Project Features	10
8.1	FastAPI Deployment	11
8.2	Voice Query	11
8.3	Image Upload and OCR	11
8.4	Report Generator Agent	12
9	LangSmith Monitoring	12
10	Team Contributions	13
10.1	Mohmad Al-Hmoud	14
10.2	Shreef Tamer	14

11 Development Workflow	15
11.1 Phase One: Retrieval and Analysis	15
11.2 Phase Two: Answer Generation	15
11.3 Phase Three: Final System Features	15
12 Challenges	15
12.1 Retrieval Quality	15
12.2 Insufficient Evidence	16
12.3 Complex Analysis	16
12.4 System Observability	16
13 Results	16
14 Future Work	17
15 Conclusion	17
16 References	17

1. Introduction

Large Language Models (LLMs) are capable of understanding and generating natural language, but they can still face challenges when they need to work with large collections of documents, retrieve precise evidence, perform calculations, compare information, or provide reliable citations.

To address these challenges, this project introduces a multi-agent architecture in which different agents are responsible for different stages of the information processing pipeline.

Instead of relying on a single agent to perform every operation, the system divides the task into three main responsibilities:

1. Retrieving relevant information.
2. Analyzing and validating the retrieved evidence.
3. Generating a final answer based on the verified evidence.

This separation of responsibilities makes the system more modular and allows each agent to specialize in a particular task.

2. Problem Statement

Traditional question-answering systems based only on LLMs may produce inaccurate answers when the required information is not directly available in the model's knowledge or when the question requires precise evidence from specific documents.

Several challenges must therefore be addressed:

- Finding relevant information from a large document collection.
- Handling ambiguous or poorly formulated user queries.
- Combining semantic and keyword-based retrieval.
- Filtering and ranking retrieved information.
- Performing accurate numerical calculations.
- Extracting and analyzing tabular information.
- Comparing information from multiple documents.
- Avoiding unsupported answers when evidence is insufficient.
- Providing accurate citations for the final response.

The proposed multi-agent system addresses these challenges by assigning specific responsibilities to specialized agents.

3. Project Objectives

The main objectives of the project are:

1. Build a multi-agent architecture for intelligent information retrieval and analysis.
2. Develop a Retriever Agent capable of using multiple retrieval strategies.
3. Develop an Analyst Agent capable of processing and analyzing retrieved evidence.
4. Develop an Answer Agent capable of generating clear and citation-supported answers.
5. Allow the Analyst Agent to request additional evidence when the retrieved information is insufficient.
6. Provide users with multiple input methods, including text, voice, and images.
7. Generate reports containing answers or analytical results.
8. Monitor the system using LangSmith.
9. Deploy the system using FastAPI.

4. System Architecture

The proposed system consists of three main intelligent agents:

1. Retriever Agent
2. Analyst Agent
3. Answer Agent

The general workflow can be represented as follows:

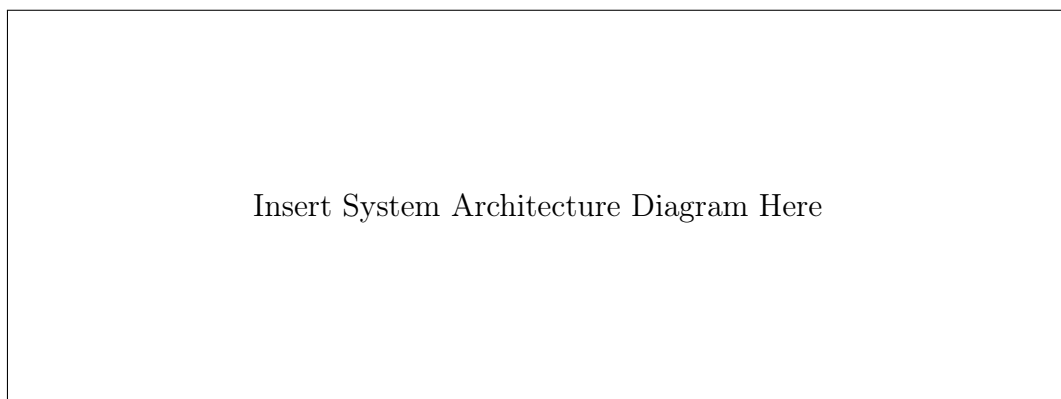
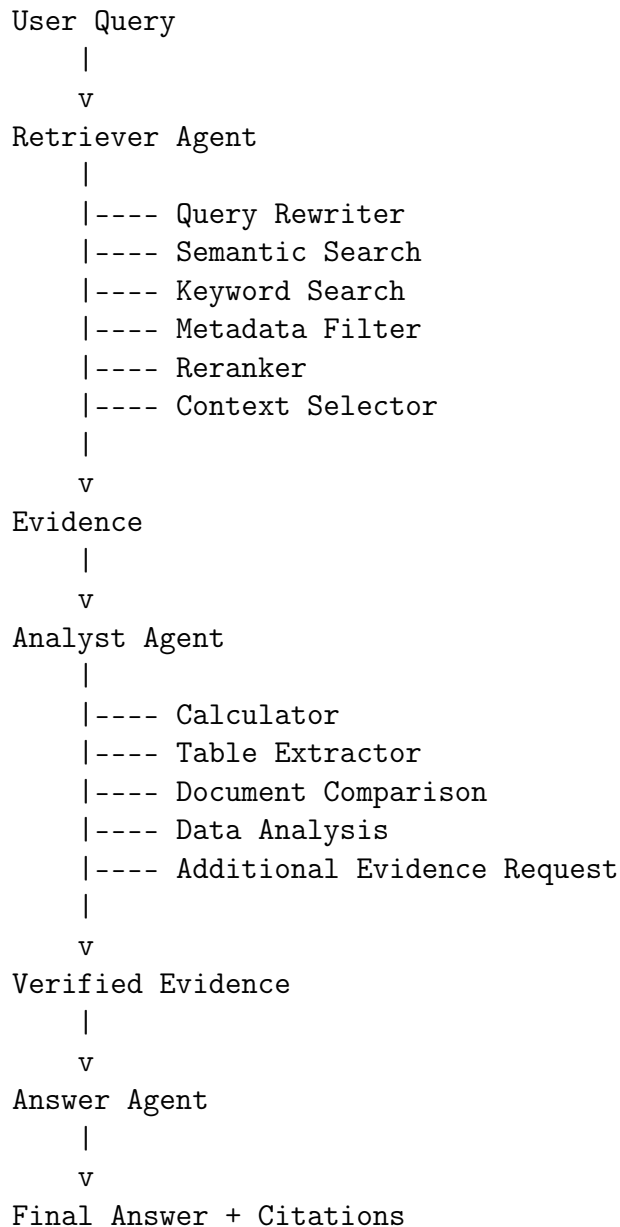


Figure 1: High-level architecture of the Coding Agents system.

The overall processing pipeline follows this structure:



5. Retriever Agent

The Retriever Agent is responsible exclusively for finding relevant information from the Vector Database. Its responsibility is retrieval rather than answer generation.

The agent uses several tools to improve retrieval quality.

5.1. Query Rewriter

The Query Rewriter reformulates the user's question into a clearer and more searchable query.

This is particularly useful when the original question contains ambiguous references or pronouns.

For example, a question such as:

“What are its disadvantages?”

may not contain enough information for an effective search. The Query Rewriter can transform it into a more explicit query based on the previous context.

5.2. Semantic Search

Semantic Search retrieves documents according to their meaning rather than only matching exact words.

The process relies on embeddings, where both the query and stored documents are represented as vectors. Similarity between these vectors is then used to identify relevant information.

5.3. Keyword Search

Keyword Search is used when exact terms are important.

This is particularly useful for:

- Technical terminology
- Names
- Specific identifiers
- Mathematical expressions
- Exact phrases

5.4. Metadata Filter

Metadata filtering allows the retrieval process to consider additional information associated with documents.

Examples include:

- Document name
- Page number
- Chapter
- Section

This allows the system to restrict retrieval to specific parts of the knowledge base when required.

5.5. Reranker

The Reranker receives an initial set of retrieved results and reorders them according to their relevance to the query.

For example, the initial retrieval stage may return 20–50 results. The Reranker then identifies which results are most closely related to the question.

5.6. Context Selector

The Context Selector selects the most useful pieces of information from the ranked results.

Instead of passing every retrieved document to the Analyst Agent, the system selects a smaller number of high-quality and diverse contexts.

Typically, approximately 5–8 useful chunks can be selected while preserving their source information for later citation.

6. Analyst Agent

The Analyst Agent receives evidence from the Retriever Agent and performs deeper analysis before an answer is generated.

Its primary responsibility is to determine what the evidence means and whether it is sufficient to answer the user’s question.

6.1. Calculator

The Calculator performs accurate numerical operations instead of relying on the LLM to estimate mathematical results.

Examples include:

- Averages
- Percentages
- Differences
- Other numerical calculations

6.2. Table Extractor

The Table Extractor extracts structured information from tables contained within documents.

For example, a research paper may contain a table with:

- Accuracy

- Precision
- Recall
- F1-score

The extracted table can then be processed and analyzed programmatically.

6.3. Document Comparison

The Document Comparison tool allows the system to compare information from multiple documents.

For example, the system can compare three research papers and identify:

- Similarities
- Differences
- Methods
- Results
- Conclusions

6.4. Data Analysis

The Data Analysis capability allows the Analyst Agent to process numerical or tabular information.

Possible operations include:

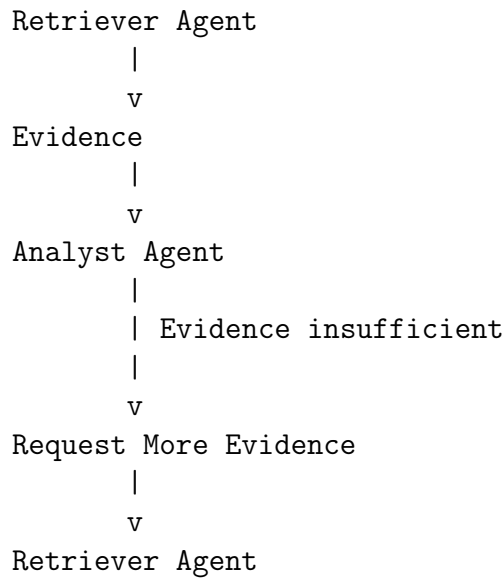
- Ranking results
- Identifying trends
- Finding the best result
- Comparing numerical values

6.5. Feedback Loop

One of the most important features of the Analyst Agent is its ability to request additional evidence.

If the retrieved information is insufficient, the Analyst Agent does not simply generate an unsupported answer.

Instead, it sends a request back to the Retriever Agent:



This feedback loop improves the reliability of the system and reduces the risk of generating answers without sufficient supporting evidence.

7. Answer Agent

The Answer Agent is responsible for generating the final response after the Analyst Agent determines that sufficient evidence is available.

Its main responsibilities are:

- Organizing the verified information.
- Generating a clear natural-language response.
- Presenting results in tables when appropriate.
- Including accurate citations.
- Referencing the original document and page number when available.

The Answer Agent therefore acts as the final communication layer between the multi-agent system and the user.

8. Final Project Features

The final stage of the project expanded the system beyond the core multi-agent architecture.

8.1. FastAPI Deployment

The system was prepared for deployment using FastAPI.

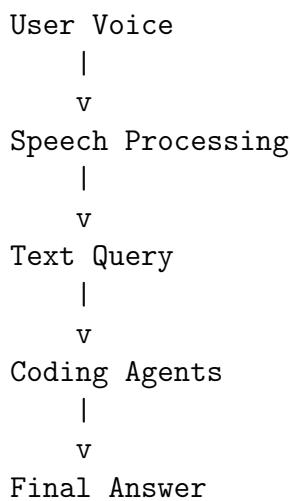
FastAPI provides an API layer through which external applications or a user interface can communicate with the Coding Agents system.

The API can receive user requests and return the generated results.

8.2. Voice Query

A voice-query feature was added to allow users to ask questions using their voice.

The general workflow is:



This feature provides an alternative to manually typing questions.

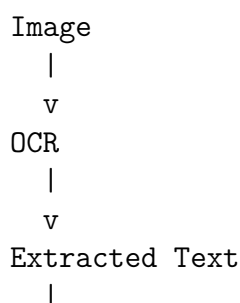
8.3. Image Upload and OCR

The system also supports image uploads.

When a user provides an image containing text, Optical Character Recognition (OCR) can be used to extract the textual content.

The extracted text can then be passed to the system as a query or additional input.

The workflow is:



```
v
Agent System
|
v
Answer
```

8.4. Report Generator Agent

A Report Generator Agent was introduced to generate PDF reports from answers or analytical results.

The agent is responsible for transforming system output into a structured report.

The report generation process may include:

- Titles
- Sections
- Tables
- Results
- References
- Formatting

An important requirement is that the Report Generator Agent has access to design-related options, allowing it to control elements such as:

- Fonts
- Font sizes
- Colors
- Layout
- Report structure

This allows generated reports to be more flexible and visually consistent.

9. LangSmith Monitoring

LangSmith was incorporated into the project to provide observability and monitoring for the LLM and agent workflows.

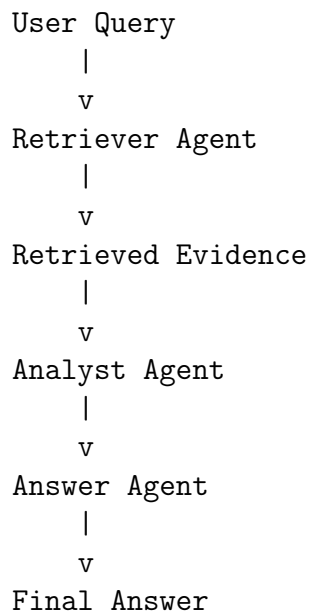
In a multi-agent system, the final answer alone does not provide enough information to understand how the system reached its result.

Monitoring allows developers to inspect the execution flow of the system.

Important information that can be monitored includes:

- Agent execution steps
- LLM calls
- Prompts and responses
- Retrieval operations
- Execution time
- Token usage
- Errors and failures

For example, when a user submits a question, the execution can be traced through the different stages:



This makes it easier to identify problems within individual components.

For example, if an answer is incorrect, monitoring can help determine whether the problem originated from retrieval, analysis, or final answer generation.

10. Team Contributions

The project was developed collaboratively by Mohmad Al-Hmoud and Shreef Tamer. The work was divided into different components to allow both team members to focus on specialized parts of the system.

10.1. Mohmad Al-Hmoud

Mohmad Al-Hmoud was primarily responsible for the following components:

- Retriever Agent.
- Query Rewriter.
- Semantic Search.
- Keyword Search.
- Metadata Filtering.
- Reranking.
- Context Selection.
- Analyst Agent.
- Calculator.
- Table Extraction.
- Document Comparison.
- Data Analysis.
- Feedback loop for requesting additional evidence.
- Report Generator Agent.
- LangSmith Monitoring.
- Preparation of the project report using LaTeX.

The main focus of this contribution was the retrieval and analysis stages, along with observability and automated report generation.

10.2. Shreef Tamer

Shreef Tamer was primarily responsible for the following components:

- Answer Agent.
- FastAPI deployment.
- Voice query functionality.
- Image upload functionality.
- OCR-based text extraction.

The main focus of this contribution was the final answer generation layer, deployment, and user-facing input capabilities.

11. Development Workflow

The project was developed in multiple stages.

11.1. Phase One: Retrieval and Analysis

The first phase focused on building the Retriever and Analyst Agents.

The Retriever Agent was responsible for finding relevant evidence, while the Analyst Agent processed the evidence and performed the required analysis.

11.2. Phase Two: Answer Generation

The second phase focused on the Answer Agent.

The Answer Agent received the processed evidence and analysis and generated the final user-facing response with appropriate citations.

11.3. Phase Three: Final System Features

The final phase focused on extending the system with additional functionalities, including:

- FastAPI deployment.
- Voice queries.
- Image uploads.
- OCR.
- Report generation.
- LangSmith monitoring.

12. Challenges

Developing a multi-agent system introduces several challenges.

12.1. Retrieval Quality

Retrieving relevant evidence is critical because the quality of the final answer depends strongly on the quality of the retrieved information.

Using multiple retrieval techniques, reranking, and context selection helps address this challenge.

12.2. Insufficient Evidence

A system may retrieve information that is relevant but insufficient to answer the complete question.

The feedback loop between the Analyst Agent and Retriever Agent addresses this problem by allowing the system to request additional evidence.

12.3. Complex Analysis

Some questions require numerical calculations, table processing, or comparison between multiple documents.

Specialized tools were therefore assigned to the Analyst Agent rather than expecting the LLM to perform every operation itself.

12.4. System Observability

Multi-agent workflows can contain many intermediate operations, making it difficult to debug the system by looking only at the final response.

LangSmith monitoring helps provide visibility into these operations.

13. Results

The final system combines retrieval, analysis, answer generation, and additional user-facing capabilities into a single multi-agent workflow.

The architecture allows different agents to specialize in different responsibilities instead of relying on a single general-purpose agent.

The implemented system provides:

- Multi-stage information retrieval.
- Evidence analysis.
- Additional evidence retrieval through a feedback loop.
- Final answer generation.
- Source-aware responses.
- Voice-based queries.
- Image-based input using OCR.
- PDF report generation.
- API-based deployment.
- LLM and agent monitoring.

14. Future Work

Several improvements can be considered for future versions of the system.

Possible improvements include:

- Improving retrieval accuracy through more advanced retrieval strategies.
- Adding more specialized analytical tools.
- Improving report customization.
- Adding more evaluation metrics for the agents.
- Developing a more advanced user interface.
- Improving monitoring and evaluation through automated feedback.
- Supporting additional input and output formats.

15. Conclusion

This project demonstrated the development of a multi-agent system for information retrieval, evidence analysis, and answer generation.

The system divides the overall task among three specialized agents. The Retriever Agent focuses on finding relevant information, the Analyst Agent processes and validates the evidence, and the Answer Agent generates the final response.

The addition of a feedback loop allows the Analyst Agent to request additional evidence when necessary, reducing the risk of unsupported answers.

The final system was further extended with FastAPI deployment, voice queries, image uploads with OCR, automated report generation, and LangSmith monitoring.

The project also provided practical experience in designing modular AI systems, integrating LLM-based agents, implementing retrieval and analysis pipelines, and monitoring complex agent workflows.

16. References

1. LangChain Documentation. <https://python.langchain.com/>
2. LangSmith Documentation. <https://docs.smith.langchain.com/>
3. FastAPI Documentation. <https://fastapi.tiangolo.com/>
4. LaTeX Documentation. <https://www.latex-project.org/>